Pyti-Lernumgebung – Benutzerhandbuch

Table of Contents

Pyti-Lernumgebung

Ausführliches Benutzerhandbuch

Für Lernende, Lehrkräfte und Administratoren Dokumentversion: 1.0
Anwendungsversion: 1.2.0 **Stand:** 25. Oktober 2026

Über dieses Handbuch

Dieses Handbuch erklärt die Pyti-Lernumgebung von der Installation bis zur Verwaltung. Es beschreibt die sichtbaren Bedienelemente, die Python-Befehle, den Spielfeldaufbau, das Lernen mit Leveln, das lokale Admin-Panel und die technische Architektur.

Die Anleitung richtet sich an:

- **Lernende**, die erste Python-Programme schreiben und Pyti steuern möchten;
- **Lehrkräfte und Begleitpersonen**, die Aufgaben vorbereiten und beim Lernen unterstützen;
- **Administratoren**, die Level, Einstellungen und Beispielprogramme ohne Änderungen am Quellcode verwalten;
- **technisch Interessierte**, die verstehen möchten, wie die Anwendung aufgebaut ist und wie sie geprüft wird.

Die Pyti-Lernumgebung ist eine lokale Desktop-Anwendung. Sie benötigt weder einen Webserver noch ein Benutzerkonto oder eine Datenbank. Sie ist ein praktisches Werkzeug für erste Programmieraufgaben, aber kein vollständiger Python-Kurs und keine sichere Ausführungsumgebung für fremde Programme.

---

1. Schnellstart

Eine erste Mission

1. Starte Pyti und wähle in der Level-Liste **„0.0.1 Der erste Beeper“**.
2. Schreibe den folgenden Code in den Editor:

   ```
   while not onBeeper():
     moveForward()

   pickBeeper()
   ```

3. Klicke auf **Start**.
4. Beobachte Pyti: Der Roboter läuft, bis er auf dem Beeper steht, und hebt ihn auf.
5. Ist das Ziel erfüllt, erscheint eine Erfolgsmeldung in der Missionsanzeige.

In diesem Programm bedeutet `while`, dass die eingerückten Anweisungen wiederholt werden. `not onBeeper()` fragt, ob Pyti **noch nicht** auf einem Beeper steht. Sobald der Sensor `True` meldet, endet die Schleife und `pickBeeper()` sammelt den Beeper ein.

Vor dem ersten Start

- Python 3.10 oder neuer muss installiert sein.
- Die Abhängigkeit PySide6 muss installiert sein.
- Für die grafische Oberfläche wird eine Desktop-Umgebung mit Qt-Unterstützung benötigt.
- Führe nur Skripte aus, deren Inhalt du kennst und denen du vertraust.

---

2. Installation und Start

Projekt vorbereiten

Öffne ein Terminal und wechsle in den Ordner mit dem Projekt. Erstelle eine virtuelle Python-Umgebung:

```
python -m venv .venv
```

Linux oder macOS

```
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Windows PowerShell

```
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Falls PowerShell das Aktivieren aus Sicherheitsgründen verweigert, kann die Umgebung stattdessen über `.\.venv\Scripts\python.exe` direkt verwendet werden. Eine Ausführungspolitik sollte nicht unüberlegt systemweit geändert werden.

Programm starten

Im Projektordner:

```
python main.py
```

Das Programm kann auch mit `python -m pyti_app` gestartet werden. Die Fensterdatei `game_window.ui`, die Beispiele und die Versionsdatei werden relativ zum Projektverzeichnis ermittelt; der Start hängt daher nicht davon ab, in welchem aktuellen Arbeitsverzeichnis das Terminal steht.

Was installiert wird

Die Abhängigkeiten stehen in `requirements.txt`. Die Anwendung benötigt PySide6 für Fenster, Widgets, Signale, Threads und die Spielfeldgrafik. Es gibt keine Datenbankmigration, keinen separaten Build-Schritt und keinen erforderlichen Online-Dienst.

Anwendung beenden

Schließe das Hauptfenster. Läuft gerade ein Skript, fordert die Anwendung den Abbruch an und schließt das Fenster nach Beendigung des Workers. Ein Skript, das in einer blockierenden nativen Operation feststeckt, kann dadurch möglicherweise nicht sofort beendet werden.

---

3. Die Benutzeroberfläche

Die Hauptansicht besteht aus einem Spielfeldbereich und einer Seitenleiste. Die genaue Anordnung kann sich abhängig von Fenstergröße und Betriebssystem leicht unterscheiden.

Spielfeldbereich

Das Spielfeld zeigt:

- das quadratische Raster;
- den Roboter Pyti mit Richtungspfeil;
- Wände und Beeper;
- das hervorgehobene Zielfeld;
- unter dem Raster die Mission des ausgewählten Levels.

Mit einem Mausklick kannst du das Spielfeld bearbeiten: **Linksklick** auf eine Zelle setzt dort einen Beeper oder entfernt einen vorhandenen. **Rechtsklick** setzt eine Wand oder entfernt sie wieder. Wände lassen sich nicht auf Pyti, einem Beeper oder dem Zielfeld platzieren. Diese Änderungen gelten zunächst nur während des aktuellen Spiels und werden nicht dauerhaft im Level gespeichert.

Seitenleiste

Level-Liste

Wähle ein Level aus, um dessen Startposition, Wände, Beeper, Richtung und Mission zu laden. Ein Levelwechsel setzt Pytis Beeper-Inventar zurück. Die Level-Liste und das Spielfeld sind während der Skriptausführung gesperrt. Mit **Level neu starten** lädst du den ausgewählten Level erneut: Pyti kehrt zum Startpunkt zurück, seine Richtung und das Inventar werden zurückgesetzt, und Wände sowie Beeper werden auf den Level-Ausgangszustand zurückgesetzt.

Editor

Im Editor schreibst du normales Python mit den Pyti-Befehlen. Er bietet:

- Zeilennummern;
- farbliche Hervorhebung von Python-Schlüsselwörtern, Zeichenfolgen, Kommentaren und Pyti-Befehlen;
- Markierung der aktuellen Zeile;
- Einfügen von vier Leerzeichen, wenn du die Tabulatortaste drückst.

Die farbliche Hervorhebung ist eine Schreibhilfe. Sie prüft nicht, ob der Code syntaktisch korrekt ist oder ob eine Variable zur Laufzeit existiert.

Wichtig: Nach einem erfolgreich abgeschlossenen Skript leert die Anwendung den Editor. Sichere längere Programme daher vorher in einer separaten Textdatei. Bei einem Skriptfehler bleibt der Inhalt in der Regel erhalten.

Beispiele

Beispiele öffnet das mitgelieferte Cookbook aus EXAMPLES.md. Es enthält kurze Muster für Schleifen, Bedingungen, Funktionen und Klassen. Die Beispielansicht ist eine Leseansicht; zum Ausprobieren kopierst du den Code manuell in den Editor oder verwendest eine im Admin-Panel gespeicherte Vorlage.

Start und Stopp

- **Start** führt den Text im Editor aus.
- **Stopp** fordert den Worker zum Abbruch auf. Während eines aktiven Laufs werden Levelauswahl, Editor, Spielfeld und Start gesperrt; Stopp bleibt verfügbar.

Während der Ausführung zeigt die Statuszeile, ob das Skript läuft, abgebrochen wurde oder beendet ist. Ein erfolgreicher Skriptablauf bedeutet nicht automatisch, dass die Mission erfüllt ist: Dafür muss zusätzlich die Zielprüfung erfolgreich sein.

Administration

Administration öffnet das lokale Verwaltungsfenster. Die Funktionen sind in Kapitel 9 ausführlich beschrieben.

---

4. Das Spielfeld verstehen

Raster und Koordinaten

Das Spielfeld besteht standardmäßig aus **15 Spalten und 15 Zeilen**. Die Koordinaten beginnen links oben bei (0, 0):

- X wächst von links nach rechts: 0 bis 14;
- Y wächst von oben nach unten: 0 bis 14.

Beispiele:

Position	Bedeutung
(0, 0)	Feld oben links
(14, 0)	Feld oben rechts
(0, 14)	Feld unten links
(14, 14)	Feld unten rechts

Richtungen

Pyti kann in vier Richtungen schauen:

Richtung	Bewegung auf dem Raster
Norden	Y wird kleiner
Osten	X wird größer
Süden	Y wird größer
Westen	X wird kleiner

Ein Schritt nach Norden vom Feld (4, 4) führt beispielsweise nach (4, 3).

Wände und Spielfeldrand

Eine Wand blockiert das Betreten ihres Feldes. Der Rand ist ebenfalls eine Blockade. frontClear(), leftClear() und rightClear() prüfen sowohl Wände als auch den Rand.

Versucht ein Skript trotzdem, in eine Wand oder über den Rand zu gehen, wird die Bewegung als Fehler abgebrochen. Verwende Sensoren vor riskanten Schritten.

Beeper und Inventar

- Ein Beeper kann auf einem Spielfeldfeld liegen.
- pickBeeper() hebt einen Beeper unter Pyti auf.
- dropBeeper() legt einen Beeper aus dem Inventar auf Pytis aktuelle Position.
- Ein Beeper, der eingesammelt wurde, liegt nicht mehr auf dem Feld.
- Pyti kann mehrere Beeper im Inventar tragen.
- Das Ziel gilt nur dann als erfüllt, wenn Pyti am Ziel steht und den **Ziel-Beeper** noch im Inventar hat.

Ein auf dem Spielfeld abgelegter Ziel-Beeper zählt nicht mehr als eingesammelt. Beim Ablegen eines normalen Beepers bleibt die Ziel-Beepereintragung erhalten.

Beeper mit der Maus platzieren

Im normalen Spielmodus kannst du Beeper durch Mausklick auf ein Feld hinzufügen oder entfernen. Ein Klick schaltet den Beeper-Zustand des Feldes um. Wände sind keine gültigen Plätze für neu platzierte Beeper.

Das Spielfeld lässt sich während eines Skriptlaufs nicht verändern. Diese Mausfunktion ist für eigene Versuche gedacht und ändert den aktuellen Spielzustand, nicht dauerhaft die gespeicherte Leveldefinition.

---

5. Pyti-Befehle und Sensoren

Python unterscheidet Groß- und Kleinschreibung. Schreibe die Befehle genau wie in den Tabellen und setze die Klammern auch dann, wenn kein Argument benötigt wird.

Aktionen

Befehl	Wirkung
moveForward()	Geht ein Feld in Pytis aktueller Blickrichtung.
turnLeft()	Dreht Pyti um 90 Grad nach links.
turnRight()	Dreht Pyti um 90 Grad nach rechts.
turnAround()	Dreht Pyti um 180 Grad.
pickBeeper()	Hebt einen Beeper vom aktuellen Feld auf. Ohne Beeper entsteht ein Fehler.
dropBeeper()	Legt einen Beeper auf das aktuelle Feld. Mit leerem Inventar entsteht ein Fehler.

Sensoren

Alle folgenden Sensoren liefern den booleschen Wert `True` oder `False`.

Sensor	True, wenn …
onBeeper()	auf dem Feld unter Pyti ein Beeper liegt.
beeperAhead()	direkt vor Pyti ein Beeper auf

Sensor	True, wenn …
	einem erreichbaren Feld liegt.
frontClear()	das Feld direkt vor Pyti nicht durch Wand oder Rand blockiert ist.
leftClear()	das Feld links relativ zu Pytis Blickrichtung frei ist.
rightClear()	das Feld rechts relativ zu Pytis Blickrichtung frei ist.
anyBeeperInBag()	Pyti mindestens einen Beeper im Inventar trägt.

beeperAhead() ist kein Ersatz für frontClear(): Ein Beeper auf einem blockierten Feld wird nicht als erreichbar gemeldet.

Kleine Befehlsfolgen

Geradeausgehen und abbiegen

```
moveForward()
turnLeft()
moveForward()
```

Pyti geht ein Feld vorwärts, dreht sich nach links und geht in der neuen Blickrichtung weiter.

Beeper nur bei Bedarf aufnehmen

```
if onBeeper():
  pickBeeper()
```

Die eingerückte Aktion wird nur ausgeführt, wenn der Sensor True liefert.

Bis zu einer Blockade gehen

```
while frontClear():
  moveForward()
```

Vor jedem Durchlauf wird geprüft, ob das nächste Feld frei ist. Wenn Pyti bereits auf einer Linie bis zum Rand blickt, endet die Schleife am letzten freien Feld.

---

6. Python lernen mit Pyti

Die Anwendung führt regulären Python-Code aus und stellt darin die Pyti-Funktionen bereit. Das macht sie geeignet, grundlegende Programmierkonzepte durch sichtbare Rückmeldungen zu üben.

Lernstufen

1. Anweisungen in Reihenfolge

Beginne mit einzelnen Befehlen:

```
moveForward()
moveForward()
pickBeeper()
```

Lernziel: Computer führen Anweisungen genau in der vorgegebenen Reihenfolge aus.

2. Wiederholungen

Verwende Schleifen, um wiederkehrenden Code nicht mehrfach schreiben zu müssen:

```
for schritt in range(4):
    moveForward()
```

Oder wiederhole abhängig vom Spielfeldzustand:

```
while frontClear():
    moveForward()
```

Lernziel: Eine feste Anzahl von Wiederholungen von einer zustandsabhängigen Schleife unterscheiden.

3. Bedingungen

```
if frontClear():
    moveForward()
else:
    turnRight()
```

Lernziel: Ein Programm kann abhängig von Sensorwerten verschiedene Wege nehmen.

4. Variablen

```
gesammelte_beeper = 0


if onBeeper():
```

```
    pickBeeper()
    gesammelte_beeper = gesammelte_beeper + 1

print(gesammelte_beeper)
```

Lernziel: Werte speichern, verändern und in Ausgaben verwenden. `print()` schreibt in die Programmkonsole, nicht in eine eigene Ergebnisbox im Spielfeld.

5. Eigene Funktionen

```
def gehe_bis_zur_blockade():
    while frontClear():
        moveForward()

gehe_bis_zur_blockade()
```

Lernziel: Wiederverwendbare Aufgaben mit einem Namen versehen und den Programmablauf strukturieren.

6. Klassen und Objekte

Fortgeschrittene Lernende können Pytis Funktionen auch aus selbst geschriebenen Klassen aufrufen:

```
class Sammler:
    def __init__(self):
        self.anzahl = 0

    def sammle_aktuellen_beeper(self):
        if onBeeper():
            pickBeeper()
            self.anzahl += 1

sammler = Sammler()
sammler.sammle_aktuellen_beeper()
print(sammler.anzahl)
```

Lernziel: Methoden, Attribute, Objekte und `self` kennenlernen.

Häufige Python-Regeln

- Nach `if`, `else`, `while`, `for`, `def`, `class` und ähnlichen Blocküberschriften steht ein Doppelpunkt.
- Der Code innerhalb eines Blocks muss eingerückt sein.
- In diesem Editor fügt die Tabulatortaste vier Leerzeichen ein.
- Zeichenfolgen stehen in Anführungszeichen, etwa `"Hallo"`.

- Kommentare beginnen mit #.
- Namen sind case-sensitive: frontClear() ist nicht dasselbe wie frontclear().
- = weist einen Wert zu; == vergleicht zwei Werte.
- and, or und not verbinden oder verändern Bedingungen.

Problemlösungsstrategie

1. Lies die Mission vollständig.
2. Markiere gedanklich Start, Ziel, Wände und Beeper.
3. Teile den Weg in einzelne Entscheidungen auf.
4. Prüfe vor jeder Bewegung, ob die Richtung frei ist.
5. Wähle, ob eine Folge von Schritten besser als Schleife beschrieben wird.
6. Teste zunächst ein kleines Skript.
7. Beobachte Pyti und vergleiche das Ergebnis mit deiner Erwartung.
8. Ändere immer nur einen Teil und teste erneut.
9. Prüfe am Schluss, ob der Ziel-Beeper noch im Inventar ist.

---

7. Level und Lernideen

Umfang

Die Anwendung enthält **50 vorgefertigte Level**. Sie sind als fortlaufende Missionen angeordnet und variieren Startposition, Ziel, Wandanzahl und Wandmuster. In der aktuellen Sammlung gibt es 50 unterschiedliche Wandlayouts und 11 Generatorstufen.

Die Level wurden automatisiert auf Koordinatengültigkeit und Ziel-Erreichbarkeit geprüft. Erreichbarkeit bedeutet, dass es mindestens einen Weg durch freie Felder gibt. Sie bedeutet nicht, dass jedes Level genau einen Weg hat oder dass die Mission nur auf eine bestimmte Weise gelöst werden kann.

Eigene Lernprogression

Die Level lassen sich beispielsweise so einsetzen:

1. **Befehlsfolge:** Bewegungen und Drehungen einzeln ausprobieren.
2. **Wand umgehen:** Sensoren und if/else verwenden.
3. **Wege wiederholen:** while und for einsetzen.
4. **Beobachten und sammeln:** onBeeper() und pickBeeper() kombinieren.
5. **Code strukturieren:** wiederkehrende Schritte in Funktionen auslagern.
6. **Eigene Herausforderung:** Beeper im normalen Spielfeld versuchsweise platzieren.

Das ist ein Vorschlag für eine Unterrichtsreihenfolge. Die Level erzwingen keine bestimmten Python-Konstrukte und geben nicht automatisch Hinweise oder Musterlösungen.

Missionsziel

In der aktuellen Anwendung ist das maschinenlesbare Ziel ein collect_beeper-Ziel. Der Ziel-Beeper muss aufgenommen sein, Pyti muss am Ziel stehen, und der Ziel-Beeper darf nicht zurück auf dem Feld liegen. Ein Skript kann technisch erfolgreich ausgeführt sein, ohne diese Bedingungen zu erfüllen.

Eigene Level bearbeiten

Das Admin-Panel kann vorhandene Level anpassen. Es erstellt keine neuen Levelnamen und bietet keine Funktion zum Hinzufügen oder Löschen einer Levelmission. Die gespeicherten Änderungen überschreiben die Daten des entsprechenden bestehenden Levels in der lokalen Installation.

Neue Level mit dem Level-Creator erstellen

Öffne im Hauptfenster **Eigenes Level erstellen**. Der eigenständige Dialog enthält eine Vorschau des 15×15-Spielfelds und speichert mehrere Entwürfe automatisch im lokalen Anwendungskonfigurationsordner. Name und Mission können im Dialog bearbeitet werden.

Wähle das gewünschte Spielfeld-Werkzeug und bearbeite die Zellen:

- **Wand:** Rechtsklick setzt oder entfernt eine Wand.
- **Beeper:** Linksklick setzt oder entfernt einen normalen Beeper.
- **Startpunkt:** Linksklick verschiebt Pyti. Eine Wandzelle ist nicht zulässig.
- **Ziel:** Linksklick verschiebt das Zielfeld und stellt sicher, dass dort ein Beeper liegt.
- **Startblickrichtung:** Auswahl Norden, Osten, Süden oder Westen.

Start- und Zielfeld können nicht mit einer Wand belegt werden; der Ziel-Beeper kann nicht versehentlich entfernt werden. Vor dem Export prüft die Anwendung Koordinaten, Objektüberschneidungen und die Erreichbarkeit des Ziels. Ist ein Level ungültig, wird der Export abgewiesen; passe dann die Wände oder Positionen an.

Mit **Speichern & in Pyti übernehmen** wird das geprüfte Level automatisch im lokalen Anwendungskonfigurationsordner gespeichert und sofort in die Levelauswahl aufgenommen. Nach einem Neustart ist es weiterhin verfügbar; eine manuelle Änderung an pyti_app/levels.py ist nicht erforderlich. Eigene

Level erhalten den Präfix Eigenes Level: und ändern keine Standardmissionen. Wird ein Level desselben Namens bereits verwendet, fragt Pyti vor dem Ersetzen nach einer Bestätigung.

Python-Datei exportieren bleibt als optionale zusätzliche Funktion verfügbar, wenn du eine Leveldefinition als Quelltext weitergeben oder manuell in ein Projekt übernehmen möchtest. Der Quelltextexport allein registriert das Level nicht automatisch; dafür ist **Speichern & in Pyti übernehmen** gedacht. Bei einer beschädigten Entwurfsdatei zeigt der Creator einen Fehler an und überschreibt die vorhandene Datei nicht automatisch.

Ein Arbeitsblatt pro Standardlevel, mit Start, Ziel, Wandzahl, berechnetem Mindestweg und Platz für eigene Planung, steht im Levelatlas. Die druckfertige PDF enthält diesen Atlas als Anhang. Der Atlas beschreibt die ursprünglichen Leveldaten; lokal angepasste Level können davon abweichen.

---

8. Ausführen, Stoppen und Fehler verstehen

Ablauf eines Skripts

1. Der Editorinhalt wird als Python-Code kompiliert.
2. Der Code läuft in einem Hintergrundthread.
3. Wenn das Skript einen Pyti-Befehl aufruft, wird die Aktion über eine Qt-Signalbrücke im GUI-Thread ausgeführt.
4. Die Oberfläche wird nach den Aktionen aktualisiert.
5. Die Anwendung meldet Erfolg, Abbruch oder einen Fehler.
6. Bei erfolgreicher Ausführung prüft die Anwendung zusätzlich das Levelziel.

Die GUI-Signalbrücke verhindert, dass die Worker-Ausführung Qt-Widgets direkt aus einem Hintergrundthread manipuliert. Die UI-Steuerung wird währenddessen gesperrt, um widersprüchliche Eingaben zu vermeiden.

Zeitlimit und Abbruch

Das Standardzeitlimit beträgt 30 Sekunden und kann im Admin-Panel auf 1 bis 300 Sekunden eingestellt werden. Der **Stopp**-Knopf fordert einen Abbruch an. Python-Schleifen werden regelmäßig auf Abbruch und Zeitlimit geprüft.

Das Zeitlimit ist eine zusätzliche Schutzmaßnahme, aber kein universeller Unterbrecher: Eine einzelne blockierende Operation in einer nativen Python-Erweiterung kann sowohl den Stopp als auch das Zeitlimit verzögern.

Typische Fehler

Symptom	Häufige Ursache	Vorgehen
Syntaxfehler	Doppelpunkt, Klammer oder Anführungszeichen fehlt	Prüfe die Zeile und die vorhergehende Blocküberschrift.
Pyti läuft gegen eine Wand	Bewegung nicht durch Sensor abgesichert	Vor dem Schritt `frontClear()` prüfen.
Kein Beeper zum Aufheben	Pyti steht nicht auf einem Beeper	Mit `onBeeper()` prüfen oder den Weg korrigieren.
Inventar ist leer	`dropBeeper()` ohne vorheriges Aufnehmen	`anyBeeperInBag()` prüfen.
Endlosschleife	Bedingung ändert sich nie	Prüfe, ob die Schleife durch Bewegung oder eine Zustandsänderung enden kann; bei Bedarf Stopp drücken.
Kein Zielerfolg	Ziel-Beeper nicht aufgenommen oder wieder abgelegt	Prüfe Position und Inventar, nicht nur den Skriptstatus.
Fehler im Skript	Status zeigt Fehlertyp, Zeilennummer und Meldung	Prüfe die genannte Zeile und den Kontext davor; der vollständige Statustext bleibt zusätzlich als Tooltip verfügbar.

Der letzte Ausführungsfehler wird im Admin-Panel für die laufende Sitzung angezeigt. Während der Ausführung zeigt das Hauptfenster bei einem Fehler Fehlertyp, Skriptzeile (wenn Python sie bereitstellt) und Meldung direkt in der Statusanzeige; der Tooltip enthält denselben Text. Die Anwendung führt kein dauerhaftes Fehlerprotokoll und bietet keinen interaktiven Debugger.

---

9. Admin-Panel

Öffnen und Zweck

Klicke im Hauptfenster auf **Administration**. Das Admin-Panel ist für die lokale Verwaltung der Lernumgebung vorgesehen. Es benötigt kein Passwort und ist keine Sicherheitsgrenze gegenüber anderen Benutzern des Computers.

Im Admin-Panel gibt es die Bereiche **Systemstatus**, **Einstellungen**, **Level bearbeiten** und **Beispielskripte**. Änderungen werden über **Speichern** dauerhaft übernommen. **Abbrechen** verwirft noch nicht gespeicherte Einstellungen und Leveländerungen.

Systemstatus

Die Statusseite zeigt die Anwendungs-Version und die Anzahl der Level sowie unterschiedliche Wandlayouts. **Prüfungen erneut ausführen** startet Selbstprüfungen.

Prüfbereich	Was geprüft wird
Module	Ob ausgewählte zentrale Anwendungsmodule importierbar sind.
Spiellogik	Sensoren, Bewegung, Beeper-Inventar und Zielprüfung in einem kleinen isolierten Testfeld.
Level	Ob 50 Level vorliegen, Werte gültig sind und Ziele ohne Wandblockade erreichbar sind.
Oberfläche	Ob zentrale Widgets einschließlich Admin-Knopf vorhanden sind.
Konfiguration	Ob beim Programmstart eine fehlerhafte Admin-Datei festgestellt wurde.
Letzter Skriptfehler	Der letzte aufgezeichnete Skriptfehler dieser Sitzung oder der Hinweis, dass keiner vorliegt.

Die Selbstprüfung ist kein vollständiger automatisierter Testlauf. Sie prüft eine Auswahl wichtiger Funktionen und zeigt die Erreichbarkeit der Level an. Sie ersetzt weder die vollständige Testsuite noch einen manuellen End-to-End-Test.

Statuswerte:

- **OK:** die jeweilige Prüfung ist erfolgreich;
- **FEHLER:** die Prüfung hat eine Abweichung festgestellt;
- **WARNUNG:** beispielsweise konnte eine gespeicherte Konfiguration nicht geladen werden.

Nach der Korrektur eines Fehlers kannst du die Prüfung erneut ausführen. Ein Fehler beim letzten Skript kann weiterhin angezeigt werden, bis ein neuer Fehler ihn ersetzt oder die Anwendung neu gestartet wird.

Einstellungen

Maximale Skriptlaufzeit

- Zulässiger Bereich: **1 bis 300 Sekunden**.
- Standardwert: **30 Sekunden**.
- Ein kürzeres Zeitlimit beendet lange Schleifen schneller.
- Ein längeres Zeitlimit kann für längere Lernprogramme erforderlich sein.
- Das Zeitlimit kann keine blockierende native Operation garantiertermaßen unterbrechen.

Pause zwischen Aktionen

- Zulässiger Bereich: **0 bis 2000 Millisekunden**.
- Standardwert: **200 Millisekunden**.
- Eine größere Pause macht Bewegungen besser beobachtbar, verlangsamt aber lange Wege.
- 0 ms lässt die Aktionen ohne zusätzliche Sichtbarkeitsverzögerung aufeinander folgen.

Die Einstellungen werden nach dem Speichern sofort für die folgenden Skriptausführungen verwendet.

Level bearbeiten

Wähle links einen bestehenden Levelnamen aus. Die editierbaren Werte sind:

- **Missionsbeschreibung:** Text, der unter dem Spielfeld angezeigt wird;
- **Start X / Y:** Startkoordinaten;

- **Startblickrichtung:** Norden, Osten, Süden oder Westen;
- **Ziel X / Y:** Position des Ziel-Beepers;
- **Wände:** Liste von Koordinaten; Einträge können hinzugefügt oder entfernt werden;
- **Beeper:** Liste von Koordinaten; Einträge können hinzugefügt oder entfernt werden.

Koordinaten reichen von 0 bis 14, gemessen vom linken oberen Feld. Wähle einen Eintrag aus, um dessen Koordinaten in den Eingabefeldern anzuzeigen. Entferne den ausgewählten Eintrag über die Schaltfläche zum Entfernen.

Regeln bei Änderungen

Vor dem Speichern prüft die Anwendung unter anderem:

- gültige Koordinaten und Blickrichtung;
- Start und Ziel liegen nicht in einer Wand;
- Beeper liegen nicht in Wänden;
- der Ziel-Beeper ist in der Beeperliste enthalten;
- der Zieltyp bleibt ein Sammelziel;
- es gibt einen Weg vom Start zum Ziel.

Ungültige Änderungen werden mit einer Fehlermeldung abgewiesen. Korrigiere die Werte und speichere erneut. Das Admin-Panel bietet keine Schaltfläche zum Erstellen, Umbenennen oder Löschen einer Levelmission.

Nach dem erfolgreichen Speichern werden geänderte Level sofort in der laufenden Anwendung aktualisiert. Ist das bearbeitete Level aktiv, wird es mit den gespeicherten Start- und Spielfelddaten neu geladen.

Beispielskripte

In diesem Bereich können Vorlagen:

- neu angelegt werden;
- umbenannt und inhaltlich bearbeitet werden;
- gelöscht werden;
- über **In Editor laden** in den Haupteditor kopiert werden.

Die Vorlagen werden mit **Speichern** lokal gesichert. Das Laden einer Vorlage kopiert ihren Text in den Haupteditor. Der Code wird dadurch nicht ausgeführt; du musst im Hauptfenster anschließend selbst **Start** betätigen.

Eine Vorlage ist normaler Python-Code. Sie ist nicht vertrauenswürdiger als ein manuell eingegebenes Skript. Führe nur Code aus, dessen Inhalt du geprüft

hast. Das Speichern einer Vorlage prüft nicht, ob ihr Python-Code syntaktisch korrekt oder sicher ist.

---

10. Datenspeicherung und Wiederherstellung

Gespeicherte Daten

Admin-Einstellungen, angepasste Level und Skriptvorlagen werden in einer JSON-Datei im plattformspezifischen Benutzer-Konfigurationsordner gespeichert. Der konkrete Ordner wird von Qt bestimmt.

Unter Linux befindet sich die Datei typischerweise hier:

~/.config/Pyti/Lernumgebung/admin.json

Der Speicherort kann auf anderen Linux-Systemen oder bei abweichender Desktopkonfiguration variieren. Unter Windows und macOS verwendet die Anwendung jeweils den passenden Qt-Konfigurationspfad.

Backups

Um Anpassungen zu sichern:

1. Beende die Anwendung.
2. Kopiere admin.json an einen sicheren Ort.
3. Gib der Sicherung einen eindeutigen Namen und notiere, zu welcher Anwendungsversion sie gehört.

Zum Wiederherstellen beendest du die Anwendung und ersetzt die aktuelle Konfigurationsdatei durch eine zuvor gesicherte Kopie.

Fehlerhafte Konfiguration

Kann admin.json nicht geladen oder validiert werden, startet die Anwendung mit Standardwerten und ursprünglichen Leveldaten. Der Fehler wird im Admin-Panel unter **Konfiguration** angezeigt.

Sichere eine fehlerhafte Datei zunächst, bevor du sie ersetzt. Das Zurücksetzen der Datei löscht lokal angepasste Level, Einstellungen und Skriptvorlagen. Die Beispieldatei im Projektordner ist nicht die Konfigurationsdatei.

Was nicht dauerhaft gespeichert wird

- Der aktuelle Roboterstand;

- manuell per Mausklick platzierte oder entfernte Beeper im laufenden Spielfeld;
- der Inhalt des Haupteditors;
- der letzte Skriptfehler.

Diese Werte gehören zum aktuellen Lauf oder zur aktuellen Sitzung. Sichere Editorprogramme separat, wenn du sie später wiederverwenden möchtest.

---

11. Sicherheit und verantwortungsvolle Nutzung

Python-Code ist nicht eingeschränkt

Pyti führt den Editorinhalt mit `exec()` und den verfügbaren Python-Builtins aus. Das ist **keine Sandbox**. Ein Skript kann grundsätzlich versuchen, Dateien zu lesen oder zu verändern, Netzwerkzugriffe vorzunehmen oder Prozesse zu starten, sofern Betriebssystem und Python-Umgebung das erlauben.

Das gilt ebenfalls für Beispielskripte aus dem Admin-Panel. Das Speichern als Vorlage macht Code nicht sicher.

Sichere Nutzung

- Führe nur Code aus, den du selbst geschrieben oder vollständig geprüft hast.
- Kopiere keine unbekannten Skripte aus Webseiten oder Nachrichten in den Editor.
- Lass Lernende ihre Programme unter Aufsicht ausführen.
- Verwende für den Unterricht, wenn nötig, ein separates Benutzerkonto oder eine entsprechend verwaltete Arbeitsumgebung.
- Sichere wichtige Dateien unabhängig von der Pyti-Anwendung.
- Verlasse dich nicht auf das Zeitlimit als Schutz vor schädlichem Code.

Admin-Bereich

Das Admin-Panel ist absichtlich ohne Passwort zugänglich. Jede Person mit Zugriff auf die laufende Anwendung kann lokale Einstellungen und Level ändern. Es ist für Bedienkomfort gedacht, nicht für Mehrbenutzer-Zugriffskontrolle oder Manipulationsschutz.

Zeitlimit und blockierende Operationen

Das Python-Zeitlimit prüft den Ablauf zwischen Python-Zeilen. Es kann eine lange Python-Schleife abbrechen, aber eine einzelne Operation, die in einer nativen Erweiterung blockiert, möglicherweise nicht rechtzeitig unterbrechen. Der Stopp-Knopf ist daher eine Bedienhilfe und keine Garantie für sofortigen Abbruch unter allen Umständen.

---

12. Fehlersuche

Die Anwendung startet nicht

1. Prüfe die Python-Version:

   python --version

2. Aktiviere die virtuelle Umgebung.
3. Installiere die im Projekt angegebenen Abhängigkeiten:

   python -m pip install -r requirements.txt

4. Starte erneut mit python main.py.
5. Lies den vollständigen Fehlertext im Terminal.

Unter Linux ohne Desktop-Sitzung kann Qt melden, dass kein Anzeige-Backend verfügbar ist. Ein Headless-Test ist möglich mit:

QT_QPA_PLATFORM=offscreen python main.py

Dieser Modus ist für Tests gedacht und ersetzt keine normale grafische Desktop-Sitzung.

Die Oberfläche reagiert nicht wie erwartet

- Während ein Worker aktiv ist, sind mehrere Bedienelemente absichtlich gesperrt.
- Bei einer Endlosschleife auf **Stopp** klicken und warten, bis der Worker beendet ist.
- Wenn sich eine blockierende native Operation nicht lösen lässt, kann ein normales Schließen des Fensters verzögert sein.

Zielmeldung erscheint nicht

Prüfe alle Bedingungen:

1. Steht Pyti genau auf dem Ziel?
2. Wurde dort der Beeper mit `pickBeeper()` aufgenommen?
3. Trägt Pyti den Ziel-Beeper noch im Inventar?
4. Hat das Skript ohne Fehler beendet?

Die Statusmeldung „Skript erfolgreich beendet“ allein bestätigt nicht die Mission.

Leveländerung wird nicht gespeichert

- Lies die Validierungsfehlermeldung vollständig.
- Prüfe, dass Wand- und Beeperkoordinaten nicht kollidieren.
- Prüfe Start, Blickrichtung und Zielkoordinaten.
- Entferne Wände, die jeden Weg zum Ziel blockieren.
- Prüfe, dass ein Beeper auf dem Ziel liegt.
- Stelle sicher, dass der Konfigurationsordner beschreibbar ist.

Beispielvorlage erscheint nicht

Eine Vorlage wird zunächst in den Haupteditor kopiert. Sie wird erst durch **Start** ausgeführt. Falls die Vorlage nach dem Schließen der Anwendung fehlt, prüfe, ob sie im Admin-Panel gespeichert wurde und ob die Konfigurationsdatei geschrieben werden kann.

Prüfen ohne grafische Interaktion

Im Projektverzeichnis:

```
python -m compileall -q pyti_app tests *.py
QT_QPA_PLATFORM=offscreen python -m unittest discover -s tests -v
```

---

13. Architektur und Projektaufbau

Überblick

Pyti ist eine lokale Desktop-Anwendung auf Basis von PySide6. Die Implementierung ist in das Python-Paket `pyti_app/` aufgeteilt. Die Quellmodule sind bewusst klein gehalten; die Projektregel begrenzt jede Python-Datei auf höchstens 150 Zeilen.

Modulübersicht

Modul	Aufgabe
main.py	Kompatibler Startpunkt der Anwendung.
pyti_app/__main__.py	Start über python -m pyti_app.
pyti_app/application.py	Erzeugt Qt-Anwendung, lädt lokale Konfiguration und verdrahtet Objekte.
pyti_app/ui_builder.py	Baut Fenster, Seitenleiste, Editor und Spielfeldansicht auf.
pyti_app/app_controller.py	Verbindet Levelwechsel, UI-Status, Start und Stopp.
pyti_app/robot.py	Bewegung, Blickrichtung und Beeper-Inventar.
pyti_app/field_manager.py	Weltzustand, Wände, Beeper, Sensoren und Zielprüfung.
pyti_app/field_rendering.py	Zeichnet Raster, Wände, Beeper und Pyti.
pyti_app/field_view.py	Mausklicks und Qt-Ansicht des Feldes.
pyti_app/level_specs.py	Beschreibungen und Rohdaten der Level.
pyti_app/level_generator.py	Erzeugt Wege und Wandmuster.
pyti_app/levels.py	Erstellt die Levelsammlung zur Laufzeit.
pyti_app/script_runtime.py	Pyti-Befehle, Sensoren, Python-Ausführung, Abbruch und Zeitlimit.
pyti_app/script_worker.py	Hintergrundthread und Qt-Signalbrücke.
pyti_app/editor.py	Code-Editor und Zeilennummern.
pyti_app/highlighter.py	Syntaxhervorhebung im Editor.
pyti_app/admin_panel.py	Admin-Dialog und Verbindung der Verwaltungsbereiche.
pyti_app/admin_health_tab.py	Anzeige des Systemstatus.
pyti_app/admin_checks.py	Import-, Spiel-, Level- und UI-

Modul	Aufgabe
	Prüfungen.
pyti_app/admin_store.py	Laden, Validieren und Speichern von Einstellungen und Leveldaten.
pyti_app/admin_level_tab.py	Formular für Änderungen an bestehenden Leveln.
pyti_app/admin_coordinates.py	Bearbeitung von Wand- und Beeperkoordinaten.
pyti_app/admin_settings_tab.py	Formulare für Ausführungszeit und Aktionspause.
pyti_app/admin_scripts_tab.py	Verwaltung von Beispielskripten.
pyti_app/example_dialog.py	Anzeige des Cookbook-Markdown.

Die Root-Dateien commands.py, editor.py, highlighter.py, level.py, playing_field.py und pyti.py halten kurze Kompatibilitätsimporte für bisherige Importpfade bereit.

Datenfluss bei einer Bewegung

1. Ein Lernskript ruft moveForward() auf.
2. script_runtime.py erkennt den Pyti-Befehl.
3. script_worker.py übergibt die Aktion mit einem Qt-Signal an den GUI-Thread.
4. field_manager.py prüft, ob die Bewegung frei ist.
5. robot.py aktualisiert Pytis Position.
6. Die Darstellung in field_rendering.py aktualisiert die Szene.
7. Der Worker setzt das Skript mit der nächsten Anweisung fort.

So bleibt das GUI-Thread-Modell für Qt eingehalten, während die Python-Schleife in einem Worker ausgeführt wird.

Leveldaten und lokale Überschreibungen

Die mitgelieferten Leveldaten werden aus level_specs.py gelesen, anschließend durch den Generator in konkrete Levelobjekte übersetzt und in levels.py als Sammlung bereitgestellt. Das Admin-Panel speichert Änderungen als JSON-Überschreibungen. Es muss daher keine Python-Quelldatei bearbeitet werden.

UI und statische Ressourcen

game_window.ui liefert das Grundfenster. Die Oberfläche wird zur Laufzeit mit Widgets ergänzt. EXAMPLES.md wird vom Beispiel-Dialog als Markdown angezeigt. VERSION liefert die Versionsnummer im Admin-Panel.

---

14. Tests und technische Prüfung

Automatisierte Tests

Die Tests liegen im Ordner tests/:

- test_robot_field.py: Bewegung, Drehung, Kollision, Sensoren, Inventar, Wand- und Beeperlogik sowie Zielprüfung;
- test_levels.py: Anzahl und Erreichbarkeit der Levelziele;
- test_script_worker.py: GUI-Thread-Zustellung und Abbruch einer Schleife;
- test_admin.py: Konfigurationsspeicherung, Validierung, Admin-UI, Vorlagenverwaltung und Fehleranzeige;
- test_module_sizes.py: Grenze von 150 Zeilen für Python-Dateien.

Tests starten:

```
python -m unittest discover -s tests -v
```

Syntaxprüfung

```
python -m compileall -q pyti_app tests *.py
```

Grafischer Starttest

```
python main.py
```

Ein kurzer Headless-Smoke-Test unter Linux kann mit timeout durchgeführt werden:

```
timeout 3s env QT_QPA_PLATFORM=offscreen python main.py
```

Der erwartete Timeout nach wenigen Sekunden bedeutet bei diesem Test, dass die Event-Schleife gestartet blieb. Andere Fehlermeldungen müssen untersucht werden.

---

15. Glossar und Nachschlagewerk

Begriff	Erklärung
Aktion	Befehl, der Pytis Zustand verändert, zum Beispiel Bewegung oder Drehen.
Admin-Panel	Lokales Verwaltungsfenster für Prüfungen, Einstellungen, Level und Vorlagen.
Beeper	Sammelobjekt auf einem Spielfeldfeld.
Boolescher Wert	Einer von zwei Wahrheitswerten: `True` oder `False`.
GUI	Grafische Benutzeroberfläche, also Fenster, Buttons, Listen und Spielfeld.
GUI-Thread	Qt-Hauptthread, in dem Widgets sicher aktualisiert werden.
Inventar	Die von Pyti eingesammelten Beeper.
Level	Einzelne Mission mit Start, Wänden, Beepern und Ziel.
Mission	Textliche Aufgabenbeschreibung zum aktiven Level.
Sensor	Abfrage des Spielfelds, die `True` oder `False` zurückgibt.
Signalbrücke	Qt-Kommunikation zwischen Hintergrundthread und GUI-Thread.
Worker	Hintergrundthread, der den Lernskriptcode ausführt.
Ziel-Beeper	Beeper, dessen Aufnahme für die Mission erforderlich ist.

Pyti-Befehle kompakt

Bewegen: `moveForward()`
Drehen: `turnLeft()`, `turnRight()`, `turnAround()`

Beeper: pickBeeper(), dropBeeper()
Sensoren: onBeeper(), beeperAhead(), frontClear()
leftClear(), rightClear(), anyBeeperInBag()

Wichtige Projektdateien

README.md Kurzübersicht und Installation
BENUTZERHANDBUCH.md Dieses ausführliche Handbuch
EXAMPLES.md Beispielprogramme im Cookbook
ROADMAP.md Geplante und erledigte Vorhaben
CHANGELOG.md Versionsänderungen
VERSION Versionsnummer
requirements.txt Python-Abhängigkeiten
docs/ARCHITECTURE.md Kurze technische Architekturübersicht
BENUTZERHANDBUCH_LEVELATLAS.md 50 Levelkarten mit Arbeitsblättern

---

16. Arbeitsblätter und Notizen

Meine erste Lösung

Level / Mission:

Was soll Pyti tun?

Welche Sensoren brauche ich?

Mein erster Programmplan:

Was ist beim Ausführen passiert?

Was habe ich verändert und warum?

Fehleranalyse

Fehlermeldung oder unerwartetes Verhalten:

Welche Zeile oder Bedingung könnte die Ursache sein?

Meine Vermutung:

Mein Test:

Ergebnis und nächste Idee:

Eigene Unterrichtsnotizen

Anhang A – Levelkarten und Arbeitsheft

Diese 50 Arbeitsblätter beziehen sich auf die mitgelieferte Levelsammlung. Koordinaten und Wandzahlen stammen aus den Leveldaten; Mindestwege sind per Breitensuche auf dem Raster berechnet und zählen nur Bewegungen, nicht Drehbefehle. Die Anzahl der kürzesten Wege zählt alternative Bewegungsrouten mit derselben Schrittzahl. Sie ist keine Anzahl aller möglichen Skripte. Jede Karte ist als eigenständige Druckseite gedacht.

Level 01 – 0.0.1 Der erste Beeper

Mission: Gehe geradeaus und lerne moveForward(). Ziel: Erreiche den Beeper bei (4, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 0)
Startblickrichtung	Osten
Ziel	(4, 0)
Wände	0
Kürzester Rasterweg	4 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Anweisungsfolge und erste Drehungen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 02 – 0.0.2 Der erste Dreh

Mission: Verbinde Geradeauslaufen mit einer ersten Drehung. Ziel: Erreiche den Beeper bei (4, 1) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 4)
Startblickrichtung	Osten
Ziel	(4, 1)
Wände	1
Kürzester Rasterweg	7 Bewegungen
Kürzeste Bewegungsrouten	34

Lernfokus: Anweisungsfolge und erste Drehungen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 03 – 0.0.3 Eine Wand

Mission: Umgehe die einzelne Blockade. Ziel: Erreiche den Beeper bei (5, 2) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 2)
Startblickrichtung	Osten
Ziel	(5, 2)
Wände	2
Kürzester Rasterweg	5 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Anweisungsfolge und erste Drehungen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 04 – 0.0.4 Zwei Richtungen

Mission: Wechsle zwischen horizontalem und vertikalem Weg. Ziel: Erreiche den Beeper bei (6, 4) und hebe ihn auf.

Merkmal	Leveldaten
Start	(1, 1)
Startblickrichtung	Osten
Ziel	(6, 4)
Wände	3
Kürzester Rasterweg	8 Bewegungen
Kürzeste Bewegungsrouten	46

Lernfokus: Anweisungsfolge und erste Drehungen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 05 – 0.0.5 Die Ecke

Mission: Finde den Weg um die Eckbarriere. Ziel: Erreiche den Beeper bei (5, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 5)
Startblickrichtung	Osten
Ziel	(5, 0)
Wände	4
Kürzester Rasterweg	10 Bewegungen
Kürzeste Bewegungsrouten	231

Lernfokus: Anweisungsfolge und erste Drehungen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 06 – 0.0.6 Kleine Treppe

Mission: Folge dem Verlauf der Treppenwände. Ziel: Erreiche den Beeper bei (6, 3) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 8)
Startblickrichtung	Osten
Ziel	(6, 3)
Wände	5
Kürzester Rasterweg	11 Bewegungen
Kürzeste Bewegungsrouten	210

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 07 – 0.0.7 Zickzack

Mission: Schlängle dich durch die versetzten Wände. Ziel: Erreiche den Beeper bei (7, 4) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 0)
Startblickrichtung	Osten
Ziel	(7, 4)
Wände	6
Kürzester Rasterweg	11 Bewegungen
Kürzeste Bewegungsrouten	204

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 08 – 0.0.8 Die Gasse

Mission: Suche den Eingang zur freien Gasse. Ziel: Erreiche den Beeper bei (8, 6) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 6)
Startblickrichtung	Osten
Ziel	(8, 6)
Wände	7
Kürzester Rasterweg	8 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 09 – 0.0.9 Sackgasse

Mission: Erkenne die Sackgasse und nimm den Umweg. Ziel: Erreiche den Beeper bei (8, 7) und hebe ihn auf.

Merkmal	Leveldaten
Start	(1, 1)
Startblickrichtung	Osten
Ziel	(8, 7)
Wände	8
Kürzester Rasterweg	13 Bewegungen
Kürzeste Bewegungsrouten	876

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 10 – 0.0.10 Der Tunnel

Mission: Bleibe im schmalen Korridor auf Kurs. Ziel: Erreiche den Beeper bei (9, 5) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 5)
Startblickrichtung	Osten
Ziel	(9, 5)
Wände	9
Kürzester Rasterweg	9 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 11 – 0.0.11 Kreuzung

Mission: Wähle an mehreren Kreuzungen den richtigen Weg. Ziel: Erreiche den Beeper bei (9, 8) und hebe ihn auf.

Merkmal	Leveldaten
Start	(2, 2)
Startblickrichtung	Osten
Ziel	(9, 8)
Wände	10
Kürzester Rasterweg	13 Bewegungen
Kürzeste Bewegungsrouten	444

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 12 – 0.0.12 Umweg nach Norden

Mission: Umgehe die Barrieren und arbeite nach Norden. Ziel: Erreiche den Beeper bei (8, 1) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 9)
Startblickrichtung	Osten
Ziel	(8, 1)
Wände	11
Kürzester Rasterweg	16 Bewegungen
Kürzeste Bewegungsrouten	841

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 13 – 0.0.13 Umweg nach Süden

Mission: Plane einen sicheren Weg nach Süden. Ziel: Erreiche den Beeper bei (2, 10) und hebe ihn auf.

Merkmal	Leveldaten
Start	(8, 0)
Startblickrichtung	Osten
Ziel	(2, 10)
Wände	12
Kürzester Rasterweg	16 Bewegungen
Kürzeste Bewegungsrouten	1597

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 14 – 0.0.14 Das Karree

Mission: Umkreise den blockierten Innenbereich. Ziel: Erreiche den Beeper bei (10, 2) und hebe ihn auf.

Merkmal	Leveldaten
Start	(2, 2)
Startblickrichtung	Osten
Ziel	(10, 2)
Wände	13
Kürzester Rasterweg	8 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 15 – 0.0.15 Das Kreuz

Mission: Entkomme aus dem Kreuzungsbereich und erreiche den Beeper.
Ziel: Erreiche den Beeper bei (0, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(4, 4)
Startblickrichtung	Osten
Ziel	(0, 0)
Wände	14
Kürzester Rasterweg	10 Bewegungen
Kürzeste Bewegungsrouten	11

Lernfokus: Bedingungen, Sensoren und einfache Umwege.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 16 – 0.0.16 Doppelter Korridor

Mission: Navigiere durch zwei hintereinanderliegende Korridore. Ziel: Erreiche den Beeper bei (11, 8) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 2)
Startblickrichtung	Osten
Ziel	(11, 8)
Wände	15
Kürzester Rasterweg	17 Bewegungen
Kürzeste Bewegungsrouten	4004

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 17 – 0.0.17 Die Spirale

Mission: Folge dem spiralförmigen Wandmuster nach innen. Ziel: Erreiche den Beeper bei (10, 10) und hebe ihn auf.

Merkmal	Leveldaten
Start	(1, 1)
Startblickrichtung	Osten
Ziel	(10, 10)
Wände	16
Kürzester Rasterweg	20 Bewegungen
Kürzeste Bewegungsrouten	4400

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 18 – 0.0.18 Das Tor

Mission: Finde die einzige günstige Durchfahrt durch die Barriere. Ziel: Erreiche den Beeper bei (5, 1) und hebe ihn auf.

Merkmal	Leveldaten
Start	(5, 12)
Startblickrichtung	Osten
Ziel	(5, 1)
Wände	17
Kürzester Rasterweg	17 Bewegungen
Kürzeste Bewegungsrouten	350

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 19 – 0.0.19 Wand-Slalom

Mission: Weiche den versetzten Wänden im Slalom aus. Ziel: Erreiche den Beeper bei (12, 4) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 10)
Startblickrichtung	Osten
Ziel	(12, 4)
Wände	18
Kürzester Rasterweg	20 Bewegungen
Kürzeste Bewegungsrouten	2857

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 20 – 0.0.20 Das L-Labyrinth

Mission: Umgehe zwei lange L-förmige Hindernisse. Ziel: Erreiche den Beeper bei (12, 1) und hebe ihn auf.

Merkmal	Leveldaten
Start	(1, 10)
Startblickrichtung	Osten
Ziel	(12, 1)
Wände	19
Kürzester Rasterweg	22 Bewegungen
Kürzeste Bewegungsrouten	25654

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 21 – 0.0.21 Drei Kammern

Mission: Durchquere drei getrennte Wandbereiche. Ziel: Erreiche den Beeper bei (12, 8) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 0)
Startblickrichtung	Osten
Ziel	(12, 8)
Wände	20
Kürzester Rasterweg	20 Bewegungen
Kürzeste Bewegungsrouten	10208

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 22 – 0.0.22 Der Haken

Mission: Finde den Ausgang aus dem hakenförmigen Aufbau. Ziel: Erreiche den Beeper bei (1, 12) und hebe ihn auf.

Merkmal	Leveldaten
Start	(2, 2)
Startblickrichtung	Osten
Ziel	(1, 12)
Wände	21
Kürzester Rasterweg	13 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 23 – 0.0.23 Diagonale

Mission: Arbeite dich an der diagonalen Wandstruktur vorbei. Ziel: Erreiche den Beeper bei (12, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 12)
Startblickrichtung	Osten
Ziel	(12, 0)
Wände	22
Kürzester Rasterweg	24 Bewegungen
Kürzeste Bewegungsrouten	354475

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 24 – 0.0.24 Die Schleife

Mission: Verlasse die Schleife und halte auf das Ziel zu. Ziel: Erreiche den Beeper bei (13, 11) und hebe ihn auf.

Merkmal	Leveldaten
Start	(1, 1)
Startblickrichtung	Osten
Ziel	(13, 11)
Wände	23
Kürzester Rasterweg	22 Bewegungen
Kürzeste Bewegungsrouten	65775

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 25 – 0.0.25 Korridor-Netz

Mission: Wähle im Korridornetz die passende Abzweigung. Ziel: Erreiche den Beeper bei (13, 7) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 7)
Startblickrichtung	Osten
Ziel	(13, 7)
Wände	24
Kürzester Rasterweg	13 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Planen, Schleifen und systematisches Prüfen.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 26 – 0.0.26 Der Innenhof

Mission: Durchquere den Innenhof ohne an einer Wand zu enden. Ziel: Erreiche den Beeper bei (7, 1) und hebe ihn auf.

Merkmal	Leveldaten
Start	(7, 13)
Startblickrichtung	Osten
Ziel	(7, 1)
Wände	25
Kürzester Rasterweg	14 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 27 – 0.0.27 Vier Abzweigungen

Mission: Prüfe an jeder Abzweigung die freie Richtung. Ziel: Erreiche den Beeper bei (13, 13) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 0)
Startblickrichtung	Osten
Ziel	(13, 13)
Wände	26
Kürzester Rasterweg	26 Bewegungen
Kürzeste Bewegungsrouten	248988

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 28 – 0.0.28 Das Fenster

Mission: Nutze die Öffnungen im Fenster-Muster. Ziel: Erreiche den Beeper bei (12, 2) und hebe ihn auf.

Merkmal	Leveldaten
Start	(2, 12)
Startblickrichtung	Osten
Ziel	(12, 2)
Wände	27
Kürzester Rasterweg	22 Bewegungen
Kürzeste Bewegungsrouten	5122

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 29 – 0.0.29 Schlangengänge

Mission: Folge den geschwungenen Gängen bis zum Ausgang. Ziel: Erreiche den Beeper bei (13, 12) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 1)
Startblickrichtung	Osten
Ziel	(13, 12)
Wände	28
Kürzester Rasterweg	24 Bewegungen
Kürzeste Bewegungsrouten	18058

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 30 – 0.0.30 Die Ringmauer

Mission: Finde die Lücke in der Ringmauer. Ziel: Erreiche den Beeper bei (13, 7) und hebe ihn auf.

Merkmal	Leveldaten
Start	(1, 7)
Startblickrichtung	Osten
Ziel	(13, 7)
Wände	29
Kürzester Rasterweg	12 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 31 – 0.0.31 Doppelschleuse

Mission: Passiere zwei Schleusen mit jeweils einer Öffnung. Ziel: Erreiche den Beeper bei (13, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 13)
Startblickrichtung	Osten
Ziel	(13, 0)
Wände	30
Kürzester Rasterweg	26 Bewegungen
Kürzeste Bewegungsrouten	426117

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 32 – 0.0.32 Das Raster

Mission: Überquere das dichte Raster Schritt für Schritt. Ziel: Erreiche den Beeper bei (13, 13) und hebe ihn auf.

Merkmal	Leveldaten
Start	(1, 1)
Startblickrichtung	Osten
Ziel	(13, 13)
Wände	31
Kürzester Rasterweg	24 Bewegungen
Kürzeste Bewegungsrouten	33013

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 33 – 0.0.33 Labyrinth Bronze

Mission: Finde den einzigen langen Weg durch das Labyrinth. Ziel: Erreiche den Beeper bei (13, 9) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 0)
Startblickrichtung	Osten
Ziel	(13, 9)
Wände	32
Kürzester Rasterweg	22 Bewegungen
Kürzeste Bewegungsrouten	21904

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 34 – 0.0.34 Labyrinth Silber

Mission: Nutze Sensoren, um Sackgassen zu erkennen. Ziel: Erreiche den Beeper bei (12, 2) und hebe ihn auf.

Merkmal	Leveldaten
Start	(2, 12)
Startblickrichtung	Osten
Ziel	(12, 2)
Wände	34
Kürzester Rasterweg	22 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 35 – 0.0.35 Labyrinth Nord

Mission: Steige durch mehrere verschachtelte Korridore nach Norden.
Ziel: Erreiche den Beeper bei (7, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(7, 13)
Startblickrichtung	Osten
Ziel	(7, 0)
Wände	36
Kürzester Rasterweg	25 Bewegungen
Kürzeste Bewegungsrouten	41796

Lernfokus: Korridore, Kreuzungen und Fehlerbehandlung.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 36 – 0.0.36 Labyrinth Süd

Mission: Finde den Ausgang am anderen Ende des Labyrinths. Ziel: Erreiche den Beeper bei (7, 13) und hebe ihn auf.

Merkmal	Leveldaten
Start	(7, 0)
Startblickrichtung	Osten
Ziel	(7, 13)
Wände	38
Kürzester Rasterweg	25 Bewegungen
Kürzeste Bewegungsrouten	32508

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 37 – 0.0.37 Das Irrgarten-Herz

Mission: Erreiche das Zentrum des großen Irrgartens. Ziel: Erreiche den Beeper bei (7, 7) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 7)
Startblickrichtung	Osten
Ziel	(7, 7)
Wände	40
Kürzester Rasterweg	7 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 38 – 0.0.38 Verlorene Kreuzung

Mission: Orientiere dich in einem dichten Kreuzungsnetz. Ziel: Erreiche den Beeper bei (1, 1) und hebe ihn auf.

Merkmal	Leveldaten
Start	(14, 14)
Startblickrichtung	Osten
Ziel	(1, 1)
Wände	42
Kürzester Rasterweg	26 Bewegungen
Kürzeste Bewegungsrouten	68590

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 39 – 0.0.39 Die lange Schleife

Mission: Durchlaufe die lange Schleife ohne Abkürzung. Ziel: Erreiche den Beeper bei (14, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 14)
Startblickrichtung	Osten
Ziel	(14, 0)
Wände	44
Kürzester Rasterweg	28 Bewegungen
Kürzeste Bewegungsrouten	1199531

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 40 – 0.0.40 Labyrinth Gold

Mission: Finde im komplexen Muster den sicheren Zielkorridor. Ziel: Erreiche den Beeper bei (13, 1) und hebe ihn auf.

Merkmal	Leveldaten
Start	(1, 13)
Startblickrichtung	Osten
Ziel	(13, 1)
Wände	46
Kürzester Rasterweg	24 Bewegungen
Kürzeste Bewegungsrouten	240579

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 41 – 0.0.41 Das große Labyrinth

Mission: Löse das vollständige Labyrinth mit Sensoren und Schleifen. Ziel: Erreiche den Beeper bei (14, 14) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 0)
Startblickrichtung	Osten
Ziel	(14, 14)
Wände	48
Kürzester Rasterweg	28 Bewegungen
Kürzeste Bewegungsrouten	14

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 42 – 0.0.42 Meisterprüfung

Mission: Bewältige alle zuvor geübten Navigationsmuster. Ziel: Erreiche den Beeper bei (0, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(14, 14)
Startblickrichtung	Osten
Ziel	(0, 0)
Wände	50
Kürzester Rasterweg	28 Bewegungen
Kürzeste Bewegungsrouten	252

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 43 – 0.0.43 Der letzte Korridor

Mission: Finde den versteckten Durchgang durch das Endspiel. Ziel: Erreiche den Beeper bei (14, 7) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 7)
Startblickrichtung	Osten
Ziel	(14, 7)
Wände	52
Kürzester Rasterweg	14 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 44 – 0.0.44 Vier Tore

Mission: Öffne gedanklich die vier Tore des Labyrinths. Ziel: Erreiche den Beeper bei (7, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(7, 14)
Startblickrichtung	Osten
Ziel	(7, 0)
Wände	54
Kürzester Rasterweg	26 Bewegungen
Kürzeste Bewegungsrouten	30

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 45 – 0.0.45 Das Wendewerk

Mission: Plane viele Drehungen, ohne eine Sackgasse zu übersehen. Ziel: Erreiche den Beeper bei (14, 10) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 0)
Startblickrichtung	Osten
Ziel	(14, 10)
Wände	56
Kürzester Rasterweg	24 Bewegungen
Kürzeste Bewegungsrouten	1

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 46 – 0.0.46 Die Tiefen

Mission: Dringe bis in die tiefste Kammer vor. Ziel: Erreiche den Beeper bei (0, 14) und hebe ihn auf.

Merkmal	Leveldaten
Start	(14, 0)
Startblickrichtung	Osten
Ziel	(0, 14)
Wände	58
Kürzester Rasterweg	28 Bewegungen
Kürzeste Bewegungsrouten	781605

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 47 – 0.0.47 Labyrinth Diamant

Mission: Finde den Zielweg durch das nahezu vollständige Raster. Ziel: Erreiche den Beeper bei (13, 13) und hebe ihn auf.

Merkmal	Leveldaten
Start	(1, 1)
Startblickrichtung	Osten
Ziel	(13, 13)
Wände	60
Kürzester Rasterweg	24 Bewegungen
Kürzeste Bewegungsrouten	64

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 48 – 0.0.48 Der Endspurt

Mission: Halte trotz dichter Wände die Orientierung. Ziel: Erreiche den Beeper bei (14, 1) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 13)
Startblickrichtung	Osten
Ziel	(14, 1)
Wände	62
Kürzester Rasterweg	26 Bewegungen
Kürzeste Bewegungsrouten	1324718

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 49 – 0.0.49 Meisterlabyrinth

Mission: Löse das anspruchsvollste bisherige Labyrinth. Ziel: Erreiche den Beeper bei (0, 0) und hebe ihn auf.

Merkmal	Leveldaten
Start	(14, 14)
Startblickrichtung	Osten
Ziel	(0, 0)
Wände	64
Kürzester Rasterweg	28 Bewegungen
Kürzeste Bewegungsrouten	84

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Level 50 – 0.0.50 Das große Finale

Mission: Finde den Weg zum letzten Beeper und beende die Lernstrecke.
Ziel: Erreiche den Beeper bei (14, 14) und hebe ihn auf.

Merkmal	Leveldaten
Start	(0, 0)
Startblickrichtung	Osten
Ziel	(14, 14)
Wände	66
Kürzester Rasterweg	28 Bewegungen
Kürzeste Bewegungsrouten	84

Lernfokus: Komplexe Wege, robuste Schleifen und eigener Lösungsplan.

Vor dem Programmieren

1. Zeichne oder markiere Start und Ziel auf einem 15×15-Raster.
2. Markiere die Wände und nenne eine mögliche erste Entscheidung.
3. Überlege, welche Prüfung vor einer riskanten Bewegung hilft.

Mein Lösungsplan

Welche Teilstrecken oder Entscheidungen wiederholen sich?

Welche Sensoren oder Python-Konstrukte willst du verwenden?

Test und Rückblick

Was soll Pyti nach den ersten drei Aktionen tun?

Wo ist dein Programm zuerst anders gelaufen als erwartet?

Hat Pyti den Ziel-Beeper aufgenommen und trägt ihn noch?

☐ Ja ☐ Nein Was würdest du beim nächsten Versuch ändern?

\newpage

Lehrkraft- und Adminnotizen

Vor der Ausgabe kann im Admin-Panel die lokale Levelkonfiguration angepasst werden. Die hier gedruckten Koordinaten beschreiben die mitgelieferten Standardlevel; nach Änderungen können sie abweichen. Eine Anpassung der Schwierigkeit kann über die Anzahl und Lage der Wände erfolgen. Vor dem Speichern muss der Ziel-Beper erreichbar bleiben.

Beobachtungsbogen

Lernende Person	Level	Beobachtung	Nächster Schritt

Freie Notizen
